



Trip Through the Neural Networks - Zero2Neuro

Y Nguyen, Alex Mitchell,
Brad Darby, Ali Nawaz,
Connor Waldrop

Ticket 1: Domain Understanding
- Group N - CS4273 Fall 2026

Who is Zero2Neuro Built For?

Target audience and expected entry-level technical competencies

01 Intended Users

Designed for students, researchers, and domain experts.

Specifically optimized for those seeking to build and evaluate deep neural networks (DNNs) without writing extensive Python code.

The workflow acts as a "no-code toolbox" for rapid experimentation.

02 ML Concepts

Assumes little prior mathematical depth, but requires fundamental conceptual awareness:

- Inputs & outputs
- Model types & activations
- Training vs. validation
- Basic losses & metrics

03 Technical Baseline

Comfort with configuration files and basic terminal execution is required:

- **Config Files:** Defining experiment parameters via small, text-based structures.
- **Terminal Tasks:** Directory navigation, editing files, and launching baseline commands on a server.

How Zero2Neuro Configuration Files Structure Experiments

Separate configuration settings define data, architecture, and training with strong structural dependencies



01 Core Representation

Experiments are split across distinct, modular configurations:

- **Data Handling:** Defines input/output data pathways and preprocessing setups.
- **Network Architecture:** Controls model type, layer sizes, and activation structures.
- **Workflow:** Dictates training, evaluation loops, and final performance reporting.



02 Configured Elements

Detailed arguments construct the functional execution pipeline:

- **Models:** Specific architecture-specific parameters.
- **Activations:** Layer-by-layer activation choices.
- **Loss & Metrics:** Defined validation/training objectives.
- **Processes:** Cross-validation, backend logging, and automated reporting formats.



03 Dependencies

Available arguments do not exist in isolation; they are deeply coupled:

- **Output-Driven Rules:** Output type directly dictates valid choices for activations, losses, and metrics.
- **Type-Specific Settings:** Selected network model type determines which architectural arguments are relevant or ignored.

Pain Points in Manual Configuration Creation

Key hurdles, errors, and inefficiencies users face when manually editing experiment files



01 Complex Coupling

Interdependent choices form a high-stakes, fragile network:

- **Overwhelming Options:** Navigating the sheer number of configuration combinations breeds systematic fatigue.
- **Cascade Effects:** Modifying a single value forces manual updates across multiple unrelated sections to prevent run-time collapse.



02 Config Mismatches

Invalid settings run but silently sabotage performance:

- **Pathological Combos:** Choosing wrong activation, loss, or metric pairings generates poor mathematical convergence.
- **Silent Failures:** Incorrect input/output shapes, data columns, or file paths trigger fatal crashes late in execution.



03 Knowledge Gap

No-code architecture still demands structural mastery:

- **Hidden ML Dependency:** Stripping Python coding does not eliminate the need for foundational ML knowledge.
- **Blind Configuration:** Setting up correct pipelines requires users to intuitively grasp how model types dictate valid architectural parameters.

Functional Requirements for the Zero2Neuro UI

Core interface capabilities designed to simplify configuration creation, validation, and execution



01 Direct Config Control

Core workflow actions to manage underlying configuration structures:

- **Graphical Workflow:** Create, edit, save, and load standard Zero2Neuro configuration files without writing raw code.
- **File Interoperability:** Import existing files, visually inspect parameter networks, and export compliant production settings.



02 Dynamic Parameter

Adaptive control surfaces that guide valid execution setups:

- **Core Variable Exposure:** Manage data input/output paths, models, layer counts, activations, losses, and metrics.
- **Context-Sensitive Layouts:** Dynamically show or hide fields based on primary selections (e.g., model types or output requirements).



03 Pre-Flight Validation

Intelligent safety nets built to eliminate runtime exceptions:

- **Multi-Level Audits:** Warn users about missing fields, empty structures, or invalid data directory targets instantly.
- **Conflict Resolution:** Flag incompatible activation/loss configurations before the user attempts pipeline execution.

Guiding Interface & Usability Requirements

Balancing ease-of-use with educational clarity to promote correct Zero2Neuro practices



01 Guided Transparency

Demystify configurations instead of hiding the underlying technology:

- **Visible Translation:** Show users in real-time how interactive UI selections translate into actual Zero2Neuro configuration options.
- **Educational Exposure:** Promote structural learning by keeping users connected to the backend syntax rather than presenting a purely opaque black box.



02 Scaffolded Design

Provide explicit guardrails and context to guide valid execution setups:

- **Clear Context & Explanations:** Pair inputs with descriptive labels, tooltips, and explanations defining experimental impact.
- **Sensible Defaults:** Pre-populate fields with safe, production-grade starting parameters to gently anchor novice users.



03 Active Safety Nets

Keep users within a "reasonable" set of machine learning practices:

- **Smart Validation Warnings:** Alert users instantly of incompatible hyperparameter selections or invalid activation/loss pairings.
- **Constructive Friction:** Prevent broken pipeline launches while using errors as teachable moments on foundational ML coupling rules.

References & AI Assistance Citations

Documenting software repositories, frameworks, and AI tools used for presentation development

< > 01 GitHub Repository

Primary codebase and technical foundation:

Zero2Neuro framework hosted by the Symbiotic Computing Laboratory.

c



02 ChatGPT (OpenAI)

Generative text & conceptual synthesis:

ChatGPT was utilized as an AI assistant to structure textual content, refine technical explanations, and optimize slide content flow for educational clarity.



03 Gemini in Slides

Visual design & layout assistance:

Google Slides Gemini feature was leveraged to co-create graphic layouts, design vector spacing, and align standard presentation components.